

# Sprint 1 Manual: System Architecture & Scope Definition

**Course:** E-Commerce

**Submission Platform:** GitHub Repository (/docs/SPRINT\_1.md)

**Format:** Markdown Document with Architectural Diagrams

## 1. Introduction

Sprint 1 initiates the architectural planning and domain modeling phase of the E-Commerce Software Development Life Cycle (SDLC) project. Prior to software implementation, system engineers and architects must formally define the target demographic, functional scope, underlying data structures, and technological frameworks.

The objective of this sprint is to establish the baseline system architecture and define the operational boundaries of the application.

## 2. Project Deliverables

Teams must submit a structured Markdown document in their assigned GitHub repository containing the following four required sections:

- **Target Audience & Market Focus:** User persona, core problem statement, and domain scope.
- **MVP Feature Scope Matrix:** Prioritized functional specification of core user workflows.
- **Tech Stack Selection & Justification:** Technical rationale for selected software frameworks and infrastructure.
- **Entity-Relationship Diagram (ERD):** Relational database schema design and data modeling specifications.

## 3. Specification Requirements

Teams must create a file named *SPRINT\_1.md* within the root or */docs* directory of the project repository. The document must conform to the structure below:

### Section 1: Target Audience & Market Focus

- **Primary Persona:** Define the representative user demographic for the platform (e.g., enterprise clients, small business owners, retail consumers).
- **Core Pain Point:** State the explicit operational or functional problem the platform addresses.

- **Domain Scope:** Identify the specific vertical market served (e.g., Consumer Electronics, Apparel, Digital Assets, Food Services).

## Section 2: Minimum Viable Product (MVP) Feature Scope

Specify four to six primary user workflows comprising the Minimum Viable Product (MVP). The defined scope must be feasible within the constraints of the academic semester.

Feature specifications must be structured according to the matrix format in Table 1:

| Category       | Feature Name                       | Description                                                                   | Priority   |
|----------------|------------------------------------|-------------------------------------------------------------------------------|------------|
| Authentication | User Registration & Authentication | Password hashing and JWT-based authentication mechanism.                      | High (MVP) |
| Catalog        | Product List & Search              | Product browsing interface with taxonomy-based filtering.                     | High (MVP) |
| Cart           | Cart Management                    | State persistent cart management (item addition, modification, and deletion). | High (MVP) |
| Checkout       | Order Processing                   | Mock or Stripe payment gateway integration and order object instantiation.    | High (MVP) |
| Admin          | Inventory Control                  | Administrative CRUD operations for product inventory.                         | Medium     |

## Section 3: Tech Stack Selection & Justification

Detail the proposed technology stack and provide a technical justification (2–3 sentences) evaluating the selected tools against viable alternatives.

- **Frontend Framework:** (e.g., React, Next.js, Vue.js)  
*Justification:* Architectural rationale for framework selection.
- **Backend Infrastructure:** (e.g., Django, FastAPI, Node.js/Express, Spring Boot)  
*Justification:* Evaluation of performance, scalability, and ecosystem support.
- **Database Management System:** (e.g., PostgreSQL, MySQL, MongoDB)  
*Justification:* Analysis of data integrity, schema requirements, and relational vs. non-relational trade-offs.
- **Caching & Asynchronous Processing (Optional):** (e.g., Redis, Celery)  
*Justification:* Application to session persistence or background task execution.

## Section 4: Entity-Relationship Diagram (ERD)

Formulate a relational database schema for the application domain. The schema must explicitly incorporate the following entity definitions:

- Users (Accounts)
- Products
- Categories
- Orders
- Order\_Items (Associative entity mapping Orders to Products)
- Cart / Cart\_Items

### Structural Requirements for Data Modeling:

1. **Keys:** Explicitly identify Primary Keys (PK) and Foreign Keys (FK) across all entities.
2. **Cardinality & Relationships:** Define exact relationship constraints (1:1, 1:N, N:M).
3. **Attribute Definitions:** Specify explicit SQL-compliant data types for each attribute (e.g., VARCHAR, INTEGER, DECIMAL, TIMESTAMP).

### Diagram Integration Protocols:

- Option A (Recommended): Render declarative syntax via **Mermaid.js** natively within Markdown.
- Option B: Embed a vector/raster diagram (e.g., dbdiagram.io, Figma) utilizing relative path references.

```
```mermaid
erDiagram
    USERS ||--o{ ORDERS : places
    ORDERS ||--|{ ORDER_ITEMS : contains
    PRODUCTS ||--o{ ORDER_ITEMS : ordered_in
    CATEGORIES ||--o{ PRODUCTS : categorizes

    USERS {
        int id PK
        string email
        string password_hash
    }
    PRODUCTS {
        int id PK
        string name
        decimal price
        int stock_quantity
    }
    ORDERS {
        int id PK
        int user_id FK
        decimal total_amount
        string status
    }
```

```

    }
    ORDER_ITEMS {
        int id PK
        int order_id FK
        int product_id FK
        int quantity
        decimal unit_price
    }
}

```

## 4. Submission Guidelines

1. Initialize a dedicated GitHub repository for the project team.
2. Grant administrative/collaborator permissions to the course instructor and assigned teaching assistant.
3. Commit the *SPRINT\_1.md* specification document and associated image assets to the primary branch.
4. Submit the repository URL to the Learning Management System (LMS) prior to the published deadline.

## 5. Evaluation Criteria

| Criterion                   | Weight | Evaluation Scope                                                                                    |
|-----------------------------|--------|-----------------------------------------------------------------------------------------------------|
| Architecture Clarity        | 30%    | Alignment of tech stack with problem constraints and validity of engineering trade-offs.            |
| ERD Completeness & Validity | 40%    | Accuracy of key constraints, relation mappings, cardinality specifications, and data normalization. |
| Scope Feasibility           | 20%    | Practical viability and bounding of the defined MVP scope within the project timeline.              |
| Documentation Quality       | 10%    | Adherence to Markdown formatting standards, structure clarity, and version control hygiene.         |